

Lab12 Register Allocation

1. 实验目标

1. xml2asmprog 读取 `.5-xml.asm`, 把 XML 变成 AsmProg。
2. preDataFlowPass 先做预处理, 把一些调用指令的隐含破坏寄存器补齐。
3. AsmDataFlowInfo 做 liveness analysis。
4. buildIgProg 根据 liveness 构建 interference graph。
5. coloring / simcoafrespisel 对图做简化、冻结、溢出、着色, 得到每个 temp 的颜色或 spill 结果。
6. asmprog2colored 把 temp 替换成真正寄存器, 并处理 spilled temps, 生成最终 `.colored.s`。
7. AsmProg::to_string() 把最终结构打印成汇编文本。

2. 数据结构

2.1 assem.hh

```
struct AssemInstr
```

1. assem 是模板字符串
2. dst 是这条汇编定义了哪些 temp
3. src 是这条汇编使用了哪些 temp
4. jumps 是这条指令可能跳到哪些 label
5. label 用于标签指令

2.2 asmprog.hh

1. AsmProg 表示整个程序, 包含 `std::vector<AsmFunction> functions;`
2. AsmFunction 表示一个函数, 包含函数名 `name` 和 `vector<AssemInstr> instructions`

2.3 asmdataflow.hh

这个文件定义的是 liveness analysis 的结果容器: AsmDataFlowInfo

1. AsmFunction* func 当前分析的是哪个函数
2. vector<AssemInstr>* instrs 指向这个函数的指令序列
3. map<size_t, std::set<int>> livein 每条指令入口处活跃的 temp 集合
4. map<size_t, std::set<int>> liveout 每条指令出口处活跃的 temp 集合

也就是对每条指令 i, 保存:

1. 这条指令读了谁
2. 这条指令写了谁
3. 执行前哪些值必须还活着
4. 执行后哪些值还得继续活着

3. predataflowpass.cc

给某些“看起来没有定义寄存器”的指令补齐隐含的 dst 信息。当前实现里最典型的是调用指令。

因为 `b1`、`b1x` 这类调用会破坏 caller-saved 寄存器 (`t0-t3`)，如果不把这些寄存器记到 `dst` 里，后面的 liveness 和 interference graph 就会漏边。漏边会让 allocator 以为某些值可以安全放在这些寄存器里，最后生成错误代码。

4. buildlg.cc

把 `asmdatflow` 结果变成 interference graph，也就是寄存器冲突图。

4.1 InterferenceGraph *buildlg

1. 先初始化：新建一个空的 graph 和 movePairs。
2. 遍历函数里的每条指令，对每条指令，从 `flowInfo` 里取：use, def, liveout。
3. 先把这些 temp 都登记进图里：只要某个 temp 在 use、def、liveout 里出现过，就先在 graph 里给它建一个节点。这样即使暂时没有边，它也会成为图里的一个点。
4. 处理同一条指令内部的并发冲突：当前实现里，凡是同一条指令中同时出现的多个 use，会两两加边。同理，多个 def 之间也会两两加边。这样做的目的，是避免同一条指令里多个同时参与运算的 temp 被错误地分到同一个寄存器。显然，对于一条指令中的 use 集合，他们如果用同一个寄存器存储，那么执行这条指令时，会同时用到这两个 temp，此时两个 temp 存在一处显然有误。对于 def 同理，被同时定义的多个 temp 存在同一个寄存器，会导致一些寄存器的值被覆盖掉。
5. 处理普通 interference：对于这条指令定义出来的每个 def，把它和 liveout 里的每个 temp 连边。含义是：如果某个值在这条指令之后还活着，那么 def 不能和它共用寄存器，否则这个 liveout 的 temp 存在寄存器中的值会被这条语句的 def 覆盖掉。这就是 interference graph 最核心的来源。
6. 特判 move 指令：如果当前指令是 `I_MOVE`，会把它的源和目的记成一对 move pair。这样后面 coloring 可能尝试 coalesce，尽量把 move 消掉。同时会从冲突集合里去掉 move 的 source，避免把本来可能合并的两个 temp 过早地强行加边。

因为对于普通语句，def 不能和 liveout 中的 temp 共用一个寄存器，比如 `add t2 t1 #1`，如果 `t2/t1` 共用一个寄存器，那么 `t1` 的值会被 `t2=t1+1` 覆盖掉，而又因为 `t1` 是 liveout 变量，后续被用到时就会出错。

但是对于 move 语句则不同，它不进行任何运算：`mov t2 t1`，`t1/t2` 的值是一样的，共用一个寄存器也不存在覆盖后值不对的问题。所以此时需要把初始化的冲突集合中 `conflictSet = liveout`，减去 move 语句的 use 集合。他们即使后续 liveout，也不和 move 语句的 def 集合冲突，可以使用同一个寄存器。

7. 最后构造并返回一个 `InterferenceGraph(graph, movePairs)`。

4.2 buildlgProg

它先检查 program 是否为空。然后对每个函数：

- 构造一个 `AsmDataFlowInfo`
- 调 `computeLiveness()`
- 再调 `buildlg(&func, &flowInfo)`
- 把结果放进 graphs

最终返回所有函数对应的 interference graphs。

5. coloring.cc

1. 不断尝试 `simplify()`
2. 尝试 `coalesce()`

3. 尝试 freeze()
4. 尝试 spill()
5. 只要上面任一阶段有变化，就继续循环
6. 循环结束后，执行 select() 回填颜色/确定真实 spill

5.1 bool Coloring::simplify()

找到一个度数小于k的节点，把这些节点压进 `simplifiedNodes` 栈，并从当前工作图删掉。由于该节点的邻居数量少于颜色数量，因此一旦我们为图中的其余部分上色，总会为它留下一块颜色。删除该节点及其所有边，以此简化需要处理的图。

5.2 bool Coloring::coalesce()

遍历候选的 movepair，取出有效的对，如果有一边是 MachineReg，用 George 策略。如果两边都不是，用 Briggs 策略。

1. George 判定（有预着色节点时）

条件是：对 $\text{Adj}(x)$ 里的每个 t ，满足下面二选一： t 已和 y 连接冲突边（已经冲突了， y 的合并不影响 t 的着色）；或者 $\text{degree}(t) < k$ (t 有多余颜色)。

2. Briggs 判定（两个都是普通节点）

先算 $\text{combined} = \text{Adj}(x) \cup \text{Adj}(y)$ ，再去掉 x 、 y 本身。统计其中 $\text{degree} \geq k$ 的高阶邻居数量 highDegreeCnt ，只有 $\text{highDegreeCnt} < k$ 才允许合并。也就是说没有多余颜色的邻居的个数小于 k ，这样合并后仍然留给合并节点颜色使用。

合并用类似并查集的方式处理，将 x 删除保留 y （如果有 MachineReg，固定设置为 y ）。

重写所有 movePairs：把所有 move 对中出现的 x 都改成 y ，产生自环 ($a==b$) 的 move 会被丢弃。用新集合替换旧 movePairs。

bool 返回是否有改动。

5.3 bool Coloring::freeze()

输入一整块语句、当前下标、当前 preScheduleBlock、临时编号计数器，以及一个输出参数 consumedUntil。

解决如下情况：先有一条 PTR_CALC，算出一个临时地址，后面紧跟或者稍后出现一条 LOAD 或 STORE，并且这个地址临时在块内只被这一次使用。如果满足这些条件，它就不生成中间的 add 临时，而是直接把最终访存写成带偏移形式。步骤如下：

解决“图里有低度节点，但它们都还在 move 关系里，所以 simplify 不动”的僵局。通过删除这些节点相关的 move 约束，让它们在下一轮能被 simplify 移除。

1. 找一个节点 pick，满足：
 - 不是机器寄存器
 - 度数 $< k$
 - $\text{isMove}(\text{pick})$ 为真（说明仍在 movePairs 里）
2. 遍历 movePairs，把所有和 pick 相关的 pair 收集到 eraseList
3. 把这些 pair 从 movePairs 删除
4. 如果删了至少一条，返回 $\text{changed}=\text{true}$

5.4 bool Coloring::spill()

当没有可 simplify 的低度节点、coalesce/freeze 又不能推进时，先从工作图里拿掉一个高风险节点，给剩余图腾出空间。

1. 在当前 graph 里找非机器寄存器节点，且 $\text{degree} \geq k$
2. 选度数最大的那个作为 candidate
3. 把它压入 simplifiedNodes
4. `eraseNode(candidate)` 从工作图删除
5. 返回 `changed=true`

这里只是先把节点拿掉，流程继续跑。最后是否真的 spill，要等 `select()` 时看有没有颜色可分。所以它是“潜在 spill 候选”。

5.5 bool Coloring::select()

逆序回填颜色，决定最终 spill。

1. 预着色机器寄存器。遍历 `ig->graph`，凡是机器寄存器（`num < 100`）直接设 `colors[node]=node`。这样后面普通 temp 只能绕开这些固定颜色。
2. 逆序弹栈给普通节点选色。弹出一个 node，找它在原图 `ig->graph` 中邻居已占用的颜色，放进 `forbidden`，在 `[0, k-1]` 中选第一个不在 `forbidden` 的颜色。找到就 `colors[node]=chosen`，找不到就 `spilled.insert(node)`。这里“看原图邻居”很重要：栈里恢复颜色时，要按原始冲突关系判断，不是按已删减的工作图。
3. 处理 `coalescedMoves` 的颜色传播。对每个代表点 `rep`，如果 `rep` 被 spill，则其合并进来的节点也标记 spill，否则把 `rep` 的颜色复制给所有被合并节点。
4. 覆盖性检查：遍历原图每个节点，要求“要么在 `colors`，要么在 `spilled`”。全覆盖才返回 `all_covered=true`。

6. asmprog2colored.cc

把每条指令里的 temp 改成真实寄存器，并在需要时插入 spill 访存。

输入 `AsmProg* program` 和每个函数对应的 `Coloring*`，输出新的 `AsmProg* colored`。

6.1 创建 spill slot

对每个函数，先看它的 `coloring->spilled`：

1. 统计 `spillCount`
2. 给每个 spilled temp 分一个固定偏移 `spillOffset[temp]`
3. 实现里从 `fp-40` 开始，每个 slot 占 4 字节（`40 + slot*4`）

6.2 修补函数首位栈空间

因为有 spill，需要额外栈空间。匹配并改写这几类指令：

1. `sub sp, sp, #imm`
2. `add fp, sp, #imm`
3. `sub sp, fp, #imm`
4. `add sp, sp, #imm`

每条都把 `imm` 增加 `spillCount * 4`。

6.3 重写每条普通指令的 src/dst

对每条非 label 指令：

1. 先复制成 patched
2. 清空 patched.src、patched.dst
3. 重建它们

src 重建规则

1. 机器寄存器 temp (num<100) 直接保留
2. 普通 temp 且未 spill：查 coloring->colors[temp]，替换成对应寄存器
3. spilled temp：
 - 分配 scratch (用 r9/r10 轮换)
 - 在指令前插 [ldr scratch, fp, #-offset]
 - src 里改成这个 scratch

dst 重建规则

1. 机器寄存器 temp 直接保留
2. 普通 temp 且未 spill：按颜色替换寄存器
3. spilled temp：
 - dst 改成 scratch (r9/r10)
 - 在指令后插 [str scratch, fp, #-offset]

所以一条原始指令会变成：

1. 若有 spilled src：若干前置 `ldr`
2. 主指令 (patched)
3. 若有 spilled dst：若干后置 `str`

7. git graph

